Laravel Controller & Policy Optimization

Eliminating repetition across VideoController, CodeController, DocumentController and more

---

The Problem

When you have many controllers — Videos, Codes, Documents, Subtitles — each with the same staff/student authorization rules, repeating `$this->authorize()` in every method becomes error-prone and hard to maintain. The solution is three layered patterns that work together: **authorizeResource()**, a **Base Controller**, and a **Base Policy**.

Option 1 — authorizeResource() (cleanest for standard CRUD)

One line in the constructor automatically maps all CRUD methods to the policy. No individual `$this->authorize()` calls needed anywhere in the file.

```
class VideoController extends Controller
{
public function __construct()
{
$this->authorizeResource(Video::class, 'video');
}

// No authorize() calls needed anywhere below
public function index() { ... }
public function show(Video $video) { ... }
public function store(Request $request) { ... }
public function update(Request $request, Video $video) { ... }
public function destroy(Video $video) { ... }
}
```

Laravel automatically maps each controller method to a policy method:

Controller method	Policy method called
index	viewAny
show	view
store	create
update	update
destroy	delete

Your policy just needs the matching method names:

```
class VideoPolicy
{
public function before(User $user, string $ability): bool|null
{
if ($user->role === 'staff') return true;
return null;
}

public function viewAny(User $user): bool { return true; }
public function view(User $user, Video $video): bool { return true; }
public function create(User $user): bool { return false; }
public function update(User $user, Video $video): bool { return false; }
public function delete(User $user, Video $video): bool { return false; }
}
```

Option 2 — Base Controller (best when you have many controllers)

Create a shared base controller that all your resource controllers extend. Wire `authorizeResource()` once — every child inherits it automatically.

```
// app/Http/Controllers/BaseController.php

class BaseController extends Controller
{
protected string $model;
protected string $modelParam;

public function __construct()
{
if (isset($this->model)) {
$this->authorizeResource($this->model, $this->modelParam);
}
}
}
```

Every controller becomes minimal — just declare the model and write the CRUD logic:

```
class VideoController extends BaseController
{
protected string $model = Video::class;
protected string $modelParam = 'video';

public function index() { ... }
public function show(Video $video) { ... }
}

class DocumentController extends BaseController
{
protected string $model = Document::class;
protected string $modelParam = 'document';

public function index() { ... }
public function show(Document $document) { ... }
}

class CodeController extends BaseController
{
protected string $model = Code::class;
protected string $modelParam = 'code';
// ...
}
```

One place to change authorization behaviour — all controllers benefit immediately.

Option 3 — Shared Base Policy (stops repeating policy logic)

Since all your resources share the same rule — staff can do everything, students read-only — extract that into a base policy. Individual policies only override what is genuinely different.

```
// app/Policies/BasePolicy.php

abstract class BasePolicy
{
// Staff always passes
public function before(User $user, string $ability): bool|null
{
if ($user->role === 'staff') return true;
return null;
}

// Students can always view
public function viewAny(User $user): bool { return true; }
public function view(User $user, mixed $model): bool { return true; }

// Students can never modify
public function create(User $user): bool { return false; }
public function update(User $user, mixed $model): bool { return false; }
public function delete(User $user, mixed $model): bool { return false; }
}
```

Every policy just extends it and only overrides what is different:

```php
class VideoPolicy extends BasePolicy
{
// Inherits everything — nothing to override
}

class DocumentPolicy extends BasePolicy
{
// Maybe students CAN download documents
public function download(User $user, Document $document): bool
{
return true;
}
}

class SubtitlePolicy extends BasePolicy
{
// Identical to base — nothing to override
}
```

Putting It All Together

The combination of all three options gives the cleanest structure. Adding a new resource becomes a two-file operation with minimal boilerplate.

```php
// Adding a new QuizController is just:

class QuizController extends BaseController
{
protected string $model = Quiz::class;
protected string $modelParam = 'quiz';
// Just write the CRUD logic — auth is inherited
}

class QuizPolicy extends BasePolicy
{
// Empty if rules are the same as base
}
```

Register the policy in `AuthServiceProvider`:

```php
protected $policies = [
Video::class => VideoPolicy::class,
Document::class => DocumentPolicy::class,
Code::class => CodePolicy::class,
Subtitle::class => SubtitlePolicy::class,
Quiz::class => QuizPolicy::class, // new resource — two files, done
];
```

Architecture at a Glance

Layer	Tool	Responsibility
Route	auth:sanctum middleware	Is the user logged in?

Layer	Tool	Responsibility
Route	role:student/staff middleware	Does this role have route access?
Controller	authorizeResource() via BaseController	Wires policy automatically
Policy	BasePolicy + per-model override	Record-level authorization
Validation	Form Request (validated())	Keeps validation out of controllers

Key Takeaway

Define the rules **once** in the base classes, override only when a specific resource needs different behaviour. Adding any new resource requires two minimal files — a controller that extends `BaseController` and a policy that extends `BasePolicy` — with authorization inherited automatically.

Generated by Claude · Laravel Best Practices Guide